Step 2 — Data Ingestion Pipeline into SQLite

Introduction

Modern analytics systems rely on data ingestion pipelines to move data from raw sources into structured storage systems such as databases or data warehouses.

These pipelines ensure that data is:

- validated before storage
- consistently structured
- reproducible across environments
- easily accessible for analytics queries

In production environments, ingestion pipelines are responsible for automating the movement of cleaned datasets into centralized data storage systems.

For this project, the normalized tables created in Notebook 1 will be loaded into a SQLite database.

SQLite is a lightweight relational database system that allows us to simulate how real analytics teams store structured datasets before analysis.

Objectives of This Notebook

The objective of this notebook is to build a professional data ingestion pipeline that:

- verifies the execution environment
- checks for required input files
- validates schemas
- loads cleaned CSV tables
- writes them into a SQLite database

The following tables will be ingested:

- customers
- orders
- agents
- interactions

These tables will be stored inside a database called:

customer_support.db

This database will be used in the next stages of the project for:

- SQL analytics
- feature engineering
- exploratory data analysis
- business insights

In [1]:
```
# Import required libraries

import pandas as pd
import sqlite3
import os
import logging
from datetime import datetime
```

Setting Up the Logging System

Logging is an essential component of professional data pipelines.

It allows data engineers and analysts to:

- track pipeline execution
- monitor data loading events
- debug failures
- maintain pipeline transparency

Instead of relying on print statements, production pipelines use structured logging systems.

In this notebook, we configure Python's logging module to track the ingestion process.

In [2]:
```
# Configure logging system

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s"
)

logger = logging.getLogger()
```

In [3]:
```
logger.info("Data Ingestion Pipeline Started")
```

```
2026-03-16 16:43:44,527 | INFO | Data Ingestion Pipeline Started
```

Environment Check

Before executing a data pipeline, it is important to verify the execution environment.

This includes confirming:

- the working directory
- the presence of required folders
- file access permissions

This step ensures that the pipeline runs consistently across environments.

In [4]:
```
# Check current working directory

current_path = os.getcwd()

logger.info(f"Current working directory: {current_path}")

current_path
```

```
2026-03-16 16:44:12,978 | INFO | Current working directory: C:\Users\chavi\Desktop\C
HAVI\PRojects\Agent Quality Audit Automation
```

Out[4]:
```
'C:\\Users\\chavi\\Desktop\\CHAVI\\PRojects\\Agent Quality Audit Automation'
```

Verifying Data Directory

The ingestion pipeline expects the normalized tables exported in Notebook 1 to be located in a data folder.

We verify that this directory exists before continuing.

In [5]:
```
data_folder = "data"

if os.path.exists(data_folder):
    logger.info("Data folder found.")
else:
    logger.error("Data folder not found.")
```

```
2026-03-16 16:44:37,030 | INFO | Data folder found.
```

Listing Available CSV Files

To ensure all required datasets are present, we list the files contained in the data directory.

This step helps detect:

- missing files
- incorrect filenames

- pipeline configuration issues

In [6]:
```
files = os.listdir(data_folder)

logger.info(f"Files detected in data directory: {files}")

files
```

```
2026-03-16 16:45:08,734 | INFO | Files detected in data directory: ['agents.csv', 'c
ustomers.csv', 'Customer_support_data.csv', 'interactions.csv', 'orders.csv']
```

Out[6]:
```
['agents.csv',
 'customers.csv',
 'Customer_support_data.csv',
 'interactions.csv',
 'orders.csv']
```

Loading CSV Tables

The normalized tables generated in Notebook 1 will now be loaded into memory.

Each CSV file corresponds to one relational table.

In [7]:
```
customers = pd.read_csv("data/customers.csv")
orders = pd.read_csv("data/orders.csv")
agents = pd.read_csv("data/agents.csv")
interactions = pd.read_csv("data/interactions.csv")

logger.info("CSV files successfully loaded.")
```

```
C:\Users\chavi\AppData\Local\Temp\ipykernel_12372\3307404775.py:4: DtypeWarning: Col
umns (5,6) have mixed types. Specify dtype option on import or set low_memory=False.
  interactions = pd.read_csv("data/interactions.csv")
2026-03-16 16:45:48,567 | INFO | CSV files successfully loaded.
```

Schema Validation

Before loading data into the database, it is important to validate the structure of each dataset.

Schema validation ensures that:

- expected columns are present
- column names match the data model
- data ingestion errors are avoided

This step mimics data validation checks used in production pipelines.

In [8]:
```
logger.info("Validating table schemas")

print("Customers Columns:", customers.columns.tolist())
print("Orders Columns:", orders.columns.tolist())
print("Agents Columns:", agents.columns.tolist())
print("Interactions Columns:", interactions.columns.tolist())
```

```
2026-03-16 16:46:36,450 | INFO | Validating table schemas
Customers Columns: ['unique_id', 'customer_city']
Orders Columns: ['order_id', 'order_date_time', 'item_price', 'product_category']
Agents Columns: ['agent_name', 'supervisor', 'manager', 'tenure_bucket', 'agent_shif
t']
Interactions Columns: ['unique_id', 'channel_name', 'category', 'sub_category', 'cus
tomer_remarks', 'issue_reported_at', 'issue_responded', 'connected_handling_time',
'csat_score']
```

Observations & Insights

Review the column lists above and confirm that they match the expected schema created in Notebook 1.

If any columns are missing or incorrectly named, the ingestion pipeline should be corrected before proceeding.

Creating SQLite Database

Databases are the foundation of analytics systems because they allow efficient storage, querying, and management of structured datasets.

In this step we create a SQLite database called:

`customer_support.db`

Before creating the database, we check whether it already exists to avoid accidental overwrites.

In [9]:
```
db_name = "customer_support.db"

if os.path.exists(db_name):
    logger.info("Database already exists.")
else:
    logger.info("Creating new database.")
```

```
2026-03-16 16:47:32,491 | INFO | Creating new database.
```

In [10]:
```
# Create database connection

conn = sqlite3.connect(db_name)
```

```
logger.info("Connected to SQLite database.")
```

```
2026-03-16 16:47:40,418 | INFO | Connected to SQLite database.
```

Writing Tables to SQLite Database

Now that the connection is established, we write each dataset into the database as a relational table.

This step transforms the CSV datasets into queryable database tables.

In [11]:
```
customers.to_sql("customers", conn, if_exists="replace", index=False)
logger.info("Customers table written to database")
```

```
2026-03-16 16:48:01,620 | INFO | Customers table written to database
```

In [12]:
```
orders.to_sql("orders", conn, if_exists="replace", index=False)
logger.info("Orders table written to database")
```

```
2026-03-16 16:48:16,201 | INFO | Orders table written to database
```

In [13]:
```
agents.to_sql("agents", conn, if_exists="replace", index=False)
logger.info("Agents table written to database")
```

```
2026-03-16 16:48:22,453 | INFO | Agents table written to database
```

In [14]:
```
interactions.to_sql("interactions", conn, if_exists="replace", index=False)
logger.info("Interactions table written to database")
```

```
2026-03-16 16:48:29,873 | INFO | Interactions table written to database
```

Confirming Successful Ingestion

To verify that the ingestion process completed successfully, we query the database to list all available tables.

In [15]:
```
query = "SELECT name FROM sqlite_master WHERE type='table';"

tables = pd.read_sql(query, conn)

tables
```

Out[15]:

	name
0	customers
1	orders
2	agents
3	interactions

Observations & Insights

Confirm that the following tables are present in the SQLite database:

- customers
- orders
- agents
- interactions

If these tables appear in the result above, the data ingestion pipeline has successfully loaded the datasets into the database.

Pipeline Completion

The data ingestion pipeline has successfully:

- verified the execution environment
- validated input datasets
- loaded normalized tables
- created a SQLite database
- stored relational tables for analytics

This database will serve as the central data layer for the next stages of the project, where we will perform:

- SQL-based exploratory data analysis
- feature engineering
- analytical summary table creation

In the next notebook, we will begin EDA Phase 1 using SQL and data engineering techniques.

Outcome

This notebook simulated a production-style ingestion pipeline, which is a critical component of modern analytics systems.

By storing structured data in a database, we enable:

- scalable querying

- faster analysis
- seamless integration with BI tools
- reliable data pipelines

In [ ]:

```

```